CF 710E — Generate a String

Rating: 2000

1. Statement

You want to generate a string with n letters 'a'. Initially, the text file is empty. Adding or deleting a letter 'a' costs x, and copying the entire content of the text file and duplicating it costs y.

Given n, x, and y, determine the minimum cost to generate the string.

2. Solution

It is intuitive to consider dp. However if you let dp_i be the minimum cost to generate i 'a's, there will be loops in the dependency graph, because deleting characters are allowed.

An extra observation can solve this. You won't delete right after you added a letter, because it is just wasting money. So every deletion is made right after a duplication or another deletion, making it clear that deletions can be tied to duplication, meaning one duplication can be followed by multiple deletions.

If k 'a's are already there, duplicating will give $2k$ 'a's, and there can be at most $k-1$ deletions that followed, or the resulting length of the string will be less than or equal to the initial length k, making it unworthy to perform the duplication. Therefore after each "group" of duplication with deletions, the resulting string will have a length greater than the initial length before the duplication, solving the loop issue.

The transition will be:

$$dp_i = \min\left(dp_{i-1} + x, \min_{\lceil \frac{i}{2} \rceil \le k < i}(dp_k + y + (2k - i)x)\right),$$

where the first term represents adding one 'a', and the second term represents duplicating k 'a's and then deleting some to reach i 'a's.

This approach has time complexity $O(n^2)$, so optimization is needed. Rewrite the second term into

$$\min_{\lceil \frac{i}{2} \rceil \le k < i}(dp_k + 2kx) + y - ix.$$

Maintaining $\min_{\lceil \frac{i}{2} \rceil \le k < i}(dp_k + 2kx)$ with a monotonic queue will work in $O(n)$.

Make $dp_k + 2kx$ and k into a pair. When you want to add it into the queue, iterate forward from the tail of the queue, and delete all pairs from the tail whose first element is larger than $dp_k + 2kx$, then push the current pair to the tail. Why can the pairs be discarded? Because they are less optimal by cost and their indices are smaller than k. When later dp_i is calculated, if k doesn't satisfy $k \ge \lceil \frac{i}{2} \rceil$, then that smaller index won't satisfy the condition, either, which means the pairs we deleted are completely useless compared to k. This will make the queue monotonic (non-increasing by the first element).

The minimum value will be at the head of the queue, but to make sure it satisfies the condition $k \ge \lceil \frac{i}{2} \rceil$, pairs at the head whose second element is smaller than $\lceil \frac{i}{2} \rceil$ should be deleted. Since every index k can be added into the queue and popped from the queue at most once, the overall complexity is $O(n)$.

3. Code

```cpp
#include<bits/stdc++.h>
using namespace std;
long long n,x,y,dp[10000005];
pair<long long,int> q[10000005];
int main(){
  cin>>n>>x>>y;
  dp[1]=x;
  long long head=1,tail=1;
  q[1].first=dp[1]+2*x;
  q[1].second=1;
  for(long long i=2;i≤n;i++){
    while(head≤tail&&q[head].second<(i+1)/2) head++;
    dp[i]=min(dp[i-1]+x,q[head].first+y-i*x);
    while(head≤tail&&q[tail].first>dp[i]+2*i*x) tail--;
    q[++tail].first=dp[i]+2*i*x;
    q[tail].second=i;
  }
  cout<<dp[n]<<endl;
  return 0;
}
```